

Design patterns, the big picture, Part 2: Gang-of-four classics revisited

Learn the GoF design patterns in Java code, starting with Strategy and Visitor

By Jeff Friesen, JavaWorld.com, 12/26/12

The Gang of Four's *Design Patterns: Elements of Reusable Object-Oriented Software* was a milestone in design pattern history. In Part 2 of his design patterns overview, Jeff Friesen focuses on the book and its 23 patterns. He first revisits the book's Strategy and Visitor patterns from a Java developer's perspective, then offers tips for learning all of the GoF patterns by successfully using *Design Patterns* as a reference manual. The article concludes with a summary of critiques of the GoF book and a warning about the perils of over-reliance on design patterns generally, which may be especially relevant for software developers new to programming.

In [Part 1](#) of this three-part series introducing design patterns, I referred to *Design Patterns: Elements of Reusable Object-Oriented Design*. This classic was written by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, who were collectively known as the Gang of Four. As most readers will know, *Design Patterns* presents 23 software design patterns that fit into the categories discussed in Part 1: Creational, structural, and behavioral.

GoF patterns on JavaWorld

David Geary's [Java design patterns](#) series introduced many of the GoF patterns in Java code. See the Resources section at the end of this article for a listing of Geary's articles on JavaWorld.

Design Patterns is canonical reading for software developers, but many new programmers are challenged by its reference format and scope. Each of the 23 patterns is described in detail, in a template format consisting of 13 sections, which can be a lot to digest. Another challenge for new Java developers is that the Gang of Four patterns spring from object-oriented programming, with examples based on C++ and Smalltalk, not Java code.

In this article I'll unpack two of the commonly used patterns -- [Strategy](#) and [Visitor](#) -- from a Java developer's perspective. Strategy is a fairly simple pattern that serves as an example of how to get your feet wet with the GoF design patterns generally; Visitor is more complex and intermediate in scope. I'll start with an example that should [demystify double dispatch](#), which is an important part of the Visitor pattern. Then I'll demonstrate the Visitor pattern in a compiler use case.

Following my examples here should help you explore and use the other GoF patterns for yourself. In addition, I'll offer [tips](#) for getting the most out of the Gang of Four book and conclude with a summary of [critiques](#) of using design patterns in software development. That discussion could be especially relevant to developers new to programming.

Unpacking Strategy

The *Strategy* pattern lets you define a family of algorithms such as those used for sorting, text composition, or layout management. Strategy also lets you encapsulate each algorithm in its own class and make them interchangeable. Each encapsulated algorithm is known as a *strategy*. At runtime, a client chooses the appropriate algorithm for its requirements.

What is a client?

A *client* is any piece of software that interacts with a design pattern. Although typically an object, a client could also be code within an application's `public static void main(String[] args)` method.

Unlike the [Decorator](#) pattern, which focuses on changing an object's *skin*, or appearance, Strategy focuses on changing the object's *guts*, meaning its changeable behaviors. Strategy lets you avoid using multiple conditional statements by moving conditional branches into their own strategy classes. These classes often derive from an abstract superclass, which the client references and uses to interact with a specific strategy.

From an abstract perspective, Strategy involves `Strategy`, `ConcreteStrategyx`, and `Context` types.

Strategy

Strategy provides a common interface to all supported algorithms. Listing 1 presents the Strategy interface.

Listing 1. `void execute(int x)` must be implemented by all concrete strategies

```
public interface Strategy
{
    public void execute(int x);
}
```

Where concrete strategies are not parameterized with common data, you can implement them via Java's interface feature. Where they are parameterized, you would instead declare an abstract class. For example, right-align, center-align, and justify text alignment strategies share the concept of a *width* in which to perform text alignment. So you would declare this *width* in the abstract class.

ConcreteStrategyx

Each `ConcreteStrategyx` implements the common interface and provides an algorithm implementation. Listing 2 implements Listing 1's Strategy interface to describe a specific concrete strategy.

Listing 2. `ConcreteStrategyA` executes one algorithm

```
public class ConcreteStrategyA implements Strategy
{
    @Override
    public void execute(int x)
    {
        System.out.println("executing strategy A: x = "+x);
    }
}
```

The `void execute(int x)` method in Listing 2 identifies a specific strategy. Think of this method as an abstraction for something more useful, like a specific kind of sorting algorithm (e.g., Bubble Sort, Insertion Sort, or Quick Sort), or a specific kind of layout manager (e.g., Flow Layout, Border Layout, or Grid Layout).

Listing 3 presents a second Strategy implementation.

Listing 3. ConcreteStrategyB executes another algorithm

```
public class ConcreteStrategyB implements Strategy
{
    @Override
    public void execute(int x)
    {
        System.out.println("executing strategy B: x = "+x);
    }
}
```

Context

Context provides the context in which the concrete strategy is invoked. Listings 2 and 3 show data being passed from a context to a strategy via a method parameter. Because a generic strategy interface is shared by all concrete strategies, some of them may not require all parameters. To avoid wasted parameters (especially when passing many different kinds of arguments to only a few concrete strategies), you could pass a reference to the context instead.

Instead of passing a context reference to the method, you could store it in the abstract class, making your method calls parameterless. However, the context would need to specify a more extensive interface that would include the contract for accessing context data in a uniform manner. The result, as shown in Listing 4, is a tighter coupling between strategies and their context.

Listing 4. Context is configured with a ConcreteStrategyx instance

```
class Context
{
    private Strategy strategy;

    public Context(Strategy strategy)
    {
        setStrategy(strategy);
    }

    public void executeStrategy(int x)
    {
        strategy.execute(x);
    }

    public void setStrategy(Strategy strategy)
    {
        this.strategy = strategy;
    }
}
```

The Context class in Listing 4 stores a strategy when it is created, provides a method to subsequently change the strategy, and provides another method to execute the current strategy. Except for passing a strategy to the constructor, this pattern can be seen in the [java.awt.Container](#) class, whose void `setLayout(LayoutManager mgr)` and void `doLayout()` methods specify and execute the layout manager strategy.

StrategyDemo

We need a client to demonstrate the previous types. Listing 5 presents a StrategyDemo client class.

Listing 5. StrategyDemo

```
public class StrategyDemo
{
    public static void main(String[] args)
    {
        Context context = new Context(new ConcreteStrategyA());
        context.executeStrategy(1);
        context.setStrategy(new ConcreteStrategyB());
        context.executeStrategy(2);
    }
}
```

A concrete strategy is associated with a Context instance when the context is created. The strategy can be subsequently changed via a context method call.

If you compile these classes and run StrategyDemo, you should observe the following output:

```
executing strategy A: x = 1
executing strategy B: x = 2
```

Revisiting the Visitor pattern

Visitor is the final software design pattern to appear in *Design Patterns*. Although this behavioral pattern is presented last in the book for alphabetical reasons, some believe that it should come last due to its complexity. Newcomers to Visitor often [struggle with this software design pattern](#).

As explained in *Design Patterns*, a visitor lets you add operations to classes without changing them, a bit of magic that is facilitated by the so-called [double dispatch](#) technique. In order to understand the Visitor pattern, we need first to digest double dispatch.

What is double dispatch?

Java and many other languages support *polymorphism* (many shapes) via a technique known as *dynamic dispatch*, in which a message is mapped to a specific sequence of code at runtime. Dynamic dispatch is classified as either single dispatch or multiple dispatch:

- **Single dispatch:** Given a class hierarchy where each class implements the same method (that is, each subclass overrides the previous class's version of the method), and given a variable that's assigned an instance of one of these classes, the type can be figured out only at runtime. For example, suppose each class implements method `print()`. Suppose too that one of these classes is instantiated at runtime and its variable assigned to variable `a`. When the Java compiler encounters `a.print();`, it can only verify that `a`'s type contains a `print()` method. It doesn't know which method to call. At runtime, the virtual machine examines the reference in variable `a` and figures out the actual type in order to call the right method. This situation, in which an implementation is based on a single type (the type of the instance), is known as *single dispatch*.
- **Multiple dispatch:** Unlike in single dispatch, where a single argument determines which method of that

name to invoke, *multiple dispatch* uses all of its arguments. In other words, it generalizes dynamic dispatch to work with two or more objects. (Note that the argument in single dispatch is typically specified with a period separator to the left of the method name being called, such as the *a* in `a.print()`.)

Finally, *double dispatch* is a special case of multiple dispatch in which the runtime types of two objects are involved in the call. Although Java supports single dispatch, it doesn't support double dispatch directly. But we can simulate it.

Do we over-rely on double dispatch?

Blogger Derek Greer believes that using double dispatch may indicate a design issue, which could impact an application's maintainability. Read Greer's "[Double dispatch is a code smell](#)" blog post and associated comments for details.

Simulating double dispatch in Java code

Wikipedia's entry on [double dispatch](#) provides a C++-based example that shows it to be more than function overloading. In Listing 6, I present the Java equivalent.

Listing 6. Double dispatch in Java code

```
public class DDDemo
{
    public static void main(String[] args)
    {
        Asteroid theAsteroid = new Asteroid();
        SpaceShip theSpaceShip = new SpaceShip();
        ApolloSpacecraft theApolloSpacecraft = new ApolloSpacecraft();
        theAsteroid.collideWith(theSpaceShip);
        theAsteroid.collideWith(theApolloSpacecraft);
        System.out.println();

        ExplodingAsteroid theExplodingAsteroid = new ExplodingAsteroid();
        theExplodingAsteroid.collideWith(theSpaceShip);
        theExplodingAsteroid.collideWith(theApolloSpacecraft);
        System.out.println();

        Asteroid theAsteroidReference = theExplodingAsteroid;
        theAsteroidReference.collideWith(theSpaceShip);
        theAsteroidReference.collideWith(theApolloSpacecraft);
        System.out.println();

        SpaceShip theSpaceShipReference = theApolloSpacecraft;
        theAsteroid.collideWith(theSpaceShipReference);
        theAsteroidReference.collideWith(theSpaceShipReference);
        System.out.println();

        theSpaceShipReference = theApolloSpacecraft;
        theAsteroidReference = theExplodingAsteroid;
        theSpaceShipReference.collideWith(theAsteroid);
        theSpaceShipReference.collideWith(theAsteroidReference);
    }
}

class SpaceShip
{
    void collideWith(Asteroid inAsteroid)
```

```

    {
        inAsteroid.collideWith(this);
    }
}

class ApolloSpacecraft extends SpaceShip
{
    void collideWith(Asteroid inAsteroid)
    {
        inAsteroid.collideWith(this);
    }
}

class Asteroid
{
    void collideWith(SpaceShip s)
    {
        System.out.println("Asteroid hit a SpaceShip");
    }

    void collideWith(ApolloSpacecraft as)
    {
        System.out.println("Asteroid hit an ApolloSpacecraft");
    }
}

class ExplodingAsteroid extends Asteroid
{
    void collideWith(SpaceShip s)
    {
        System.out.println("ExplodingAsteroid hit a SpaceShip");
    }

    void collideWith(ApolloSpacecraft as)
    {
        System.out.println("ExplodingAsteroid hit an ApolloSpacecraft");
    }
}

```

Listing 6 follows its C++ counterpart as closely as possible. The final four lines in the `main()` method along with the `void collideWith(Asteroid inAsteroid)` methods in `SpaceShip` and `ApolloSpacecraft` demonstrate and simulate double dispatch.

Consider the following excerpt from the end of `main()`:

```

theSpaceShipReference = theApolloSpacecraft;
theAsteroidReference = theExplodingAsteroid;
theSpaceShipReference.collideWith(theAsteroid);
theSpaceShipReference.collideWith(theAsteroidReference);

```

The third and fourth lines use single dispatch to figure out the correct `collideWith()` method (in `SpaceShip` or `ApolloSpacecraft`) to invoke. This decision is made by the virtual machine based on the type of the reference stored in `theSpaceShipReference`.

From within `collideWith()`, `inAsteroid.collideWith(this);` uses single dispatch to figure out the correct class (`Asteroid` or `ExplodingAsteroid`) containing the desired `collideWith()` method. Because `Asteroid` and `ExplodingAsteroid` overload `collideWith()`, the type of argument `this` (`SpaceShip` or `ApolloSpacecraft`) is used to distinguish the correct `collideWith()` method to call.

And with that, we have accomplished double dispatch. To recap, we first called `collideWith()` in `SpaceShip` or `ApolloSpacecraft`, and then used its argument and `this` to call one of the `collideWith()`

methods in Asteroid or ExplodingAsteroid.

When you run DDDemo, you should observe the following output:

```
Asteroid hit a SpaceShip
Asteroid hit an ApolloSpacecraft

ExplodingAsteroid hit a SpaceShip
ExplodingAsteroid hit an ApolloSpacecraft

ExplodingAsteroid hit a SpaceShip
ExplodingAsteroid hit an ApolloSpacecraft

Asteroid hit a SpaceShip
ExplodingAsteroid hit a SpaceShip

Asteroid hit an ApolloSpacecraft
ExplodingAsteroid hit an ApolloSpacecraft
```

Now that we've fully unpacked the double dispatch technique, let's return to the Visitor pattern.

Using Visitor to print and evaluate abstract syntax trees

Design Patterns uses a compiler context as the Visitor pattern's motivation. Although Visitor can be used in any hierarchical context where you don't want to mix operations on a given structure with the structure's elements, we'll follow the GoF example to study this pattern.

A compiler converts source code into executable code. The analysis phase scans characters into tokens, parses the tokens to verify syntactic correctness, and creates an abstract syntax tree (AST) that represents the program as a hierarchy of nodes.

The AST is then type-checked to verify semantic correctness (variables must be declared before they are used, for example). This data structure is also commonly used to drive code generation. These operations are often performed by visitors.

Consider an expression language involving addition, subtraction, multiplication, division, negation, floating-point numbers, and parentheses. An expression AST features nodes representing all of these items except for parentheses, which determine the tree structure.

AST visitors visit these nodes and are based on a class hierarchy that's rooted in an abstract class or interface. The root type abstracts methods for visiting the different kinds of nodes, and is extended by concrete subclasses that implement specific kinds of visitors, as shown in Figure 1.

Figure 1. The Visitor pattern is partly based on a hierarchy of visitor classes (click to enlarge)

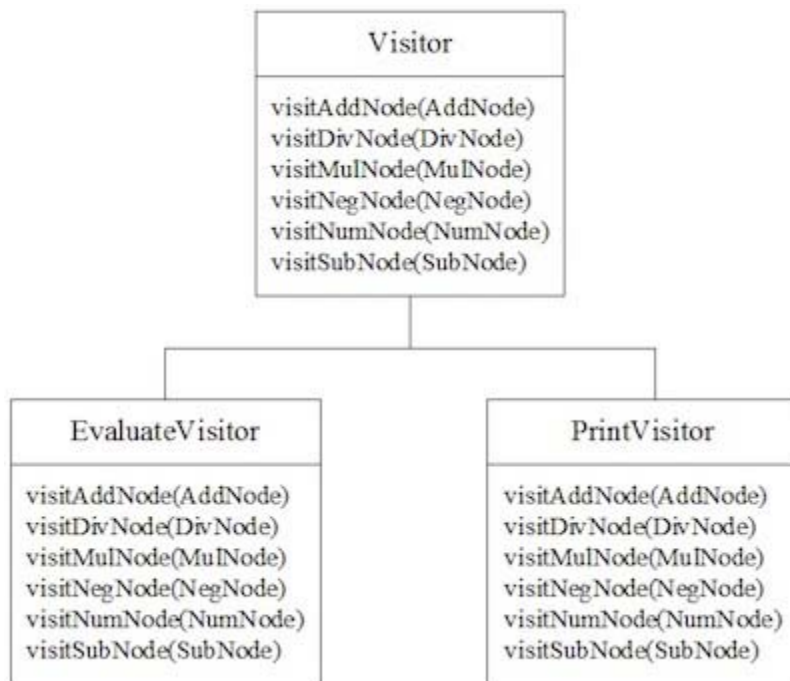


Figure 1 reveals a convention in which each visitor method begins with the word `visit` and ends with the name of the node class. Furthermore, the argument passed to this method has the node class as its type.

Each concrete AST class declares an `accept(Visitor)` method that receives a visitor class instance as its argument. This method invokes the appropriate visitor method on this argument, and passes a reference to itself as the invoked method's argument, as illustrated by Figure 2.

Figure 2. The Visitor pattern is partly based on a client-specific hierarchy of node classes

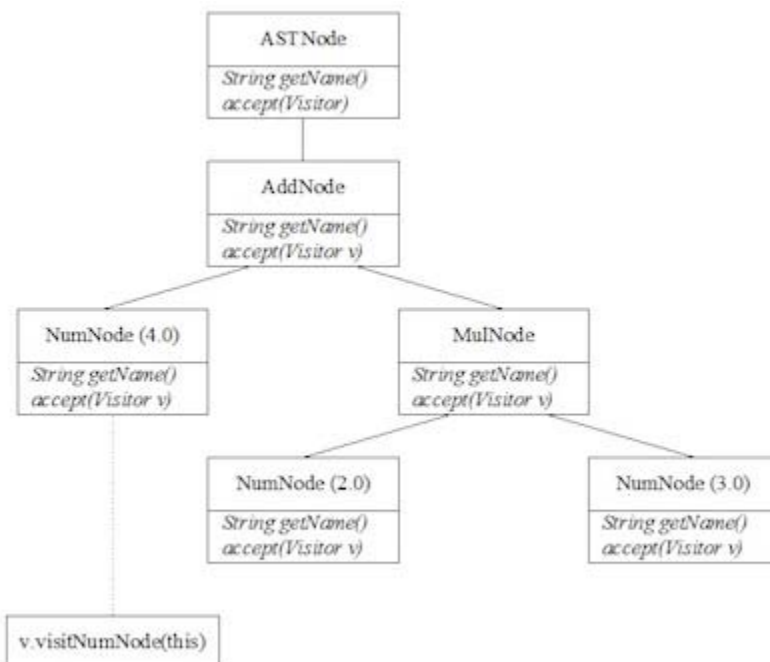


Figure 2 describes an AST for the expression: $4.0+2.0*3.0$. For brevity, only the code executed by `NumNode`'s `accept(Visitor)` method is shown. The `getName()` method is used to return the name of the node, and is only invoked by the print visitor.

Evaluating expressions via visitors

I've created a downloadable [expression evaluator application](#) that creates an AST for the expression language. The following excerpt from the application's EE class parses an expression (identified by args[0]) into an AST:

```
Parser p = new Parser();
ASTNode n = p.parse(args[0]);
```

The created AST is then visited to print its node hierarchy and evaluate its expression. For example, the excerpt below creates a print visitor and passes it to the accept(Visitor) method of the AST's root node:

```
n.accept(new PrintVisitor());
```

The application simulates double dispatch in Java code as follows:

1. Single dispatch causes the accept(Visitor) method of the appropriate ASTNode subclass instance (whose reference is stored in n) to be invoked with the PrintVisitor instance as its argument. (ASTNode is implemented as Java interface, but it could just as easily have been implemented as an abstract class.)
2. Within accept(Visitor), v.visitx(this); is executed, where x depends on the accept(Visitor) method that is called. Single dispatch causes v.visitx(x); in the appropriate Visitor subclass instance (whose reference is stored in v) to be invoked with the current node's reference as its argument.

The various visitor methods continue to traverse the tree by making accept(Visitor) calls on child nodes. For example, the following PrintVisitor class excerpt shows how PrintVisitor's visitAddNode(AddNode n) method visits its left and right child nodes:

```
public void visitAddNode(AddNode n)
{
    depth++;
    n.left.accept(this);
    depth--;
    println(n.getName());
    depth++;
    n.right.accept(this);
    depth--;
}
```

Running the expression evaluator

The following command shows you how to use EE to evaluate the expression: 4.0+2.0*3.0:

```
java EE 4.0+2.0*3.0
```

When you run this command, you should observe the following output:

```
4.0
+
      2.0
    *
      3.0

4.0+2.0*3.0 = 10.000000
```

Design Patterns tips

Now that we've looked in detail at two of the GoF patterns, let's turn our attention to the book that started it all. This section and the next will be a guide for readers new to the *Design Patterns* book. Reading the [Design patterns critique](#) in the next section may be especially helpful for newer developers.

Design Patterns is essentially a catalog of 23 software design patterns. Despite presenting a short tutorial on each pattern along with a larger case study, the book is largely laid out as a reference manual. Following a simple study plan will help you get the most out of it.

If you're new to design patterns, I recommend that you start by reading the introduction in Chapter 1. You'll learn what constitutes a design pattern, discover how design patterns are described, find out how to select and use design patterns, and more. You might want to read this chapter a couple of times to let its concepts sink in. Next, read Chapter 2, which introduces several design patterns in a document editor case study.

Pattern Hatching -- a GoF supplement

A few years after *Design Patterns* debuted, one of its authors (the late John Vlissides) wrote a supplementary volume called *Pattern Hatching*. This book focuses on putting design patterns to work, extending some of the original GoF patterns, and helping those who want to write their own patterns. Read more about this book's aims in John Vlissides's own words by visiting the Portland Pattern Repository's [Pattern Hatching](#) page.

Chapters 3, 4, and 5 comprise the catalog of patterns. Each chapter begins with an overview of the type of pattern being discussed (creational, structural, or behavioral) and then presents the patterns in alphabetical order. You probably shouldn't attempt to read through these chapters sequentially because you'll quickly get bogged down and start forgetting important details.

Instead, try studying a single chapter to learn how to distinguish between patterns of a similar type. Alternatively, you might browse from pattern to pattern. I believe that an inexperienced object-oriented designer would be well-off first studying the following eight design patterns, which are some of the simplest and most commonly used of the 23 patterns in the book. (This advice is from the readers guide that precedes Chapter 1 of *Design Patterns*, by the way.)

1. Abstract factory
2. Adapter
3. Composite
4. Decorator
5. Factory method
6. Observer
7. Strategy
8. Template method

Another option is to follow the "simple followed by intermediate followed by advanced" pattern order proposed by Web developer Michael Mahemoff in "[GoF Design Patterns: Rapid Learning Tips](#)." Finally, for the Java developer looking for Java-based examples, I recommend David Geary's [Java design patterns](#) series on JavaWorld. Geary doesn't cover all 23 patterns, so you might want to also check out James Sugrue's DZone series, [Design patterns uncovered](#).

Regardless of your approach to exploring the 23 patterns, consider summarizing and implementing each pattern (as I demonstrated with the Strategy pattern earlier in this article) for yourself. That will help you understand and retain the pattern, and will make the pattern your own, rather than letting it remain the

product of someone else's work.

Design Patterns critiqued

The GoF's *Design Patterns* book has been praised by many, but it hasn't stood the test of time without some critique. One well-known critic is [Mark Jason Dominus](#), who famously criticized GoF software design patterns in his "[Design Patterns Aren't](#)" slideshow presentation.

Dominus points out that pattern languages as introduced in Christopher Alexander's *Pattern Language* book (the architectural inspiration for software design patterns) help you decide *what* needs to be designed without telling you *how* to design it. You make up the patterns that you think will lead to good designs. In contrast, he sees the GoF's approach as discovering existing patterns and then implementing them habitually.

A participant in a StackOverflow topic addressing sources of [design pattern criticism](#) reinforced that viewpoint:

The feeling is that when we teach patterns we cause developers, especially junior developers, to try to cram all problems into the set of patterns that they have learned, which can create more obtuse and cumbersome problems.

Software developer Jeff Atwood has also critiqued the GoF patterns in a blog post entitled "[Rethinking Design Patterns](#)." Atwood says that the GoF patterns rely too heavily on boilerplate code: "[I]t's a sign that your language is fundamentally broken [when] you find yourself frequently writing a bunch of boilerplate design pattern code to deal with a 'recurring design problem,'" he writes. Atwood also finds design patterns as a whole overly complex, stating that developers should focus on simpler solutions before applying complex design pattern recipes.

Conclusion to Part 2

Software design patterns are part of a larger pattern universe. The final article in this series will broaden the usual discussion of design patterns by focusing on patterns relevant to software development, but not directly related to code. We'll look at interaction design patterns, architectural patterns, and communication and presentation patterns and discuss their relevance to contemporary software development.

In the meantime, what do you think about the Gang of Four book describing which pattern to use for a given software problem, versus the older idea that patterns should help us decide what needs to be designed, but not how to design it? Should introductory Java courses and books cover the GoF software design patterns, or are junior developers better off without them? What would help Java developers at every level better grasp when it's appropriate to use a software design pattern, and when it's not? Share your thoughts in the Comments section.